

Mini SQL Agent — Project Documentation

1. Project Overview

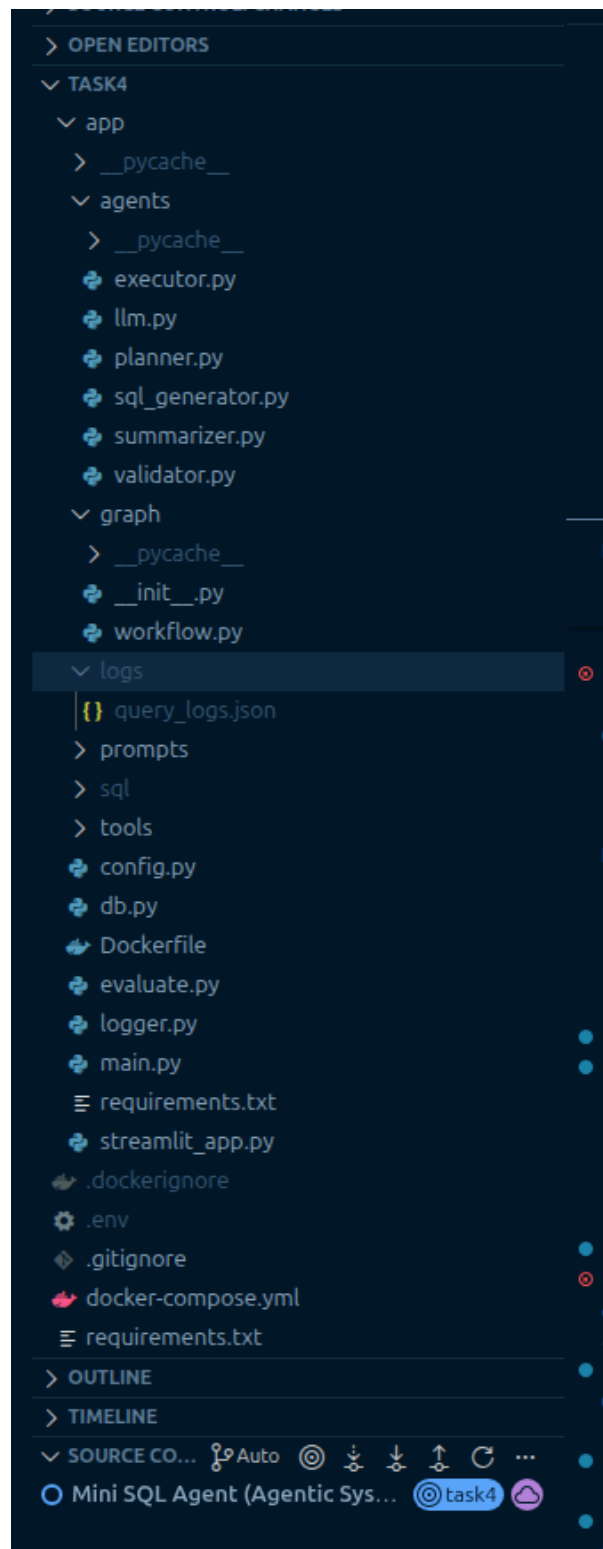
Mini SQL Agent is an agentic Text-to-SQL system that takes a natural language question, plans the query, generates SQL, executes it against PostgreSQL, self-corrects on failure, and returns a natural language summary.

Stack: FastAPI · Claude Haiku · PostgreSQL 15 · Streamlit · Docker Compose

2. Project Structure

task4/

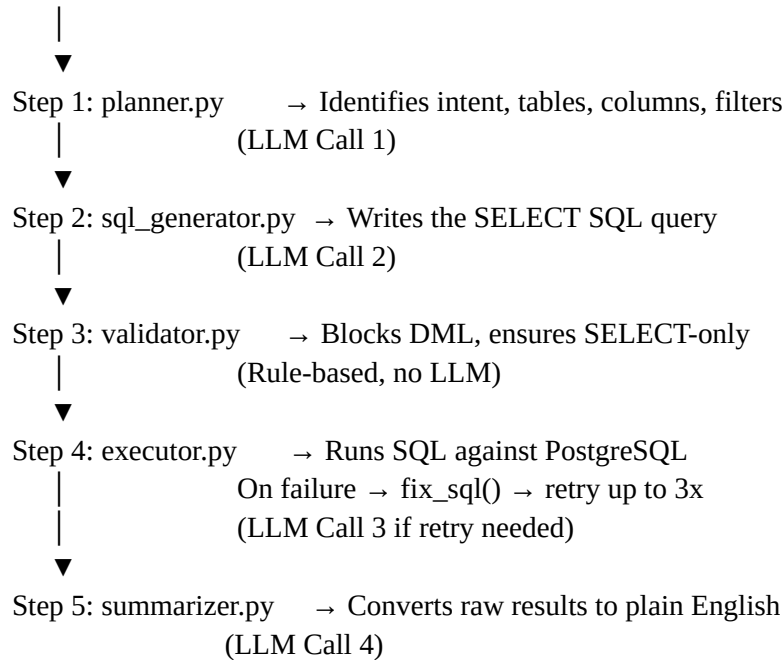
```
|— docker-compose.yml      # Orchestrates all 3 services
|— .env                    # Your secrets (never commit this)
|— .gitignore
|— app/
|   |— Dockerfile
|   |— requirements.txt
|   |— config.py           # Loads env vars
|   |— db.py               # SQLAlchemy engine
|   |— logger.py           # JSON logging to logs/
|   |— main.py             # FastAPI — POST /agent/sql
|   |— streamlit_app.py    # Chat UI
|   |— evaluate.py         # Benchmark runner
|   |— agents/
|       |— llm.py          # Anthropic client wrapper
|       |— planner.py      # LLM call 1 — understand query
|       |— sql_generator.py # LLM call 2 — generate SQL, LLM call 3 — fix SQL
|       |— validator.py    # Rule-based SELECT-only check
|       |— executor.py     # Execute SQL with retry logic
|       |— summarizer.py   # LLM call 4 — natural language summary
|   |— graph/
|       |— workflow.py     # Orchestrates the full agent pipeline
|   |— prompts/
|       |— __init__.py     # Re-exports all prompts
|       |— templates.py    # All LLM system prompts
|   |— tools/
|       |— db_tools.py     # execute_query() using SQLAlchemy
|   |— sql/
|       |— seed.sql        # classicmodels schema + data
|   |— logs/
|       |— query_logs.json # Auto-generated execution logs
```



3. Agent Flow

Every question goes through these 5 steps in order:

User Question



4. API Reference

Endpoint: POST /agent/sql Base URL: <http://localhost:8001>

Request:

```
json
{
  "question": "How many shipped orders are from USA customers?"
}
```

Response:

```
json
{
  "sql": "SELECT COUNT(*) FROM orders o JOIN customers c ...",
  "result": [{"count": 42}],
  "summary": "There are 42 shipped orders from customers in the USA.",
  "status": "success",
  "retry_count": 0,
  "latency_ms": 1234.5,
  "error": null
}
```

5. Environment Variables

File location: task4/.env

ANTHROPIC_API_KEY=sk-ant-your-key-here

DATABASE_URL=postgresql://postgres:postgres@db:5432/classicmodels

POSTGRES_USER=postgres

POSTGRES_PASSWORD=postgres

POSTGRES_DB=classicmodels

6. Running the Evaluation

```
- ./app/sql/seed.sql:/docker-entrypoint-initdb.d/seed.sql
• sakshamrimal@sakshamrimal-Latitude-5430:~/fusemachine/week3/task4$ docker-compose exec api python evaluate.py
```

Question	SQL (truncated)	Executed	Retries	Status
How many customers are th...	SELECT COUNT("customerNum...	✓	0	Success
List all offices and thei...	SELECT "officeCode", "cit...	✓	0	Success
What are the top 5 produc...	SELECT "productCode", "pr...	✓	0	Success
How many orders have stat...	SELECT COUNT("orderNumber...	✓	0	Success
Which customers are from ...	SELECT "customerNumber", ...	✓	0	Success
What is the total payment...	SELECT SUM("amount") FROM...	✓	0	Success
List employees and their ...	SELECT "employeeNumber", ...	✓	0	Success
How many shipped orders a...	SELECT COUNT("orders"."or...	✓	0	Success

```
Results:
Total queries      : 8
Success rate       : 8/8 (100%)
Queries retried    : 0
Failed queries     : 0
```

7. Logs

Every query is automatically logged to app/logs/query_logs.json.

Each entry includes: question, step name, timestamp, SQL generated, retry count, success/failure, latency.

```
app > logs > {} query_logs.json > ...
1 {"question": "How many customers are in germany?", "step": "plan_start", "timestamp": "2026-05-20T16:30:31.045974+00:00"}
2 {"question": "How many customers are in germany?", "step": "plan_complete", "plan": {"intent": "Count the number of customers located in Germany",
  "tables": ["customers"], "columns": ["customers.customerNumber", "customers.country"], "filters": ["customers.country = 'Germany'"], "aggregations":
  ["COUNT(customers.customerNumber)"], "joins": []}, "timestamp": "2026-05-20T16:30:32.674325+00:00"}
3 {"question": "How many customers are in germany?", "step": "sql_generate_start", "timestamp": "2026-05-20T16:30:32.675858+00:00"}
4 {"question": "How many customers are in germany?", "step": "sql_generate_complete", "sql": "SELECT COUNT(\\"customerNumber\\") FROM \\"customers\\" WHERE
  \\"country\\" = 'Germany'", "timestamp": "2026-05-20T16:30:34.512123+00:00"}
5 {"question": "How many customers are in germany?", "step": "execution_start", "sql": "SELECT COUNT(\\"customerNumber\\") FROM \\"customers\\" WHERE
  \\"country\\" = 'Germany'", "timestamp": "2026-05-20T16:30:34.512984+00:00"}
6 {"question": "How many customers are in germany?", "step": "execution_complete", "success": false, "retry_count": 3, "error": "Only SELECT queries are
  allowed. Query starts with: The error you're encountering ", "row_count": 0, "timestamp": "2026-05-20T16:30:38.814656+00:00"}
7 {"question": "How many customers are in germany?", "step": "workflow_complete", "success": false, "latency_ms": 7773.59, "summary": "I was unable to
  answer your question after 3 attempts. Last error: Only SELECT queries are allowed. Query starts with: The error you're encountering ", "timestamp":
  "2026-05-20T16:30:38.816200+00:00"}
8 {"question": "How many customers are in germany?", "step": "plan_start", "timestamp": "2026-05-20T16:35:20.331937+00:00"}
9 {"question": "How many customers are in germany?", "step": "plan_complete", "plan": {"intent": "Count the number of customers located in Germany", "tables":
  ["customers"], "columns": ["customers.customerNumber", "customers.country"], "filters": ["customers.country = 'Germany'"], "aggregations": ["COUNT
  (customers.customerNumber)"], "joins": []}, "timestamp": "2026-05-20T16:35:21.769557+00:00"}
10 {"question": "How many customers are in germany?", "step": "sql_generate_start", "timestamp": "2026-05-20T16:35:21.771408+00:00"}
11 {"question": "How many customers are in germany?", "step": "sql_generate_complete", "sql": "SELECT COUNT(\\"customerNumber\\") FROM \\"customers\\" WHERE
  \\"country\\" = 'Germany'", "timestamp": "2026-05-20T16:35:22.865136+00:00"}
12 {"question": "How many customers are in germany?", "step": "execution_start", "sql": "SELECT COUNT(\\"customerNumber\\") FROM \\"customers\\" WHERE \\"country\\" =
  'Germany'", "timestamp": "2026-05-20T16:35:22.866032+00:00"}
13 {"question": "How many customers are in germany?", "step": "execution_complete", "success": false, "retry_count": 3, "error": "(psycopg2.OperationalError)
  could not translate host name \\"postgres\\" to address: Name or service not known\\n\\n(Background on this error at: https://sqlalche.me/e/20/e3q8)",
  "row_count": 0, "timestamp": "2026-05-20T16:35:26.619840+00:00"}
14 {"question": "How many customers are in germany?", "step": "workflow_complete", "success": false, "latency_ms": 6289.46, "summary": "I was unable to answer
  your question after 3 attempts. Last error: (psycopg2.OperationalError) could not translate host name \\"postgres\\" to address: Name or service not
  known\\n\\n(Background on this error at: https://sqlalche.me/e/20/e3q8)", "timestamp": "2026-05-20T16:35:26.620976+00:00"}
15 {"question": "how many customers from germany?", "step": "plan_start", "timestamp": "2026-05-20T16:37:37.879291+00:00"}
16 {"question": "how many customers from germany?", "step": "plan_complete", "plan": {"intent": "Count the number of customers from Germany", "tables":
  ["customers"], "columns": ["customers.customerNumber", "customers.country"], "filters": ["country = 'Germany'"], "aggregations": ["COUNT(customerNumber)
  "], "joins": []}, "timestamp": "2026-05-20T16:37:39.270258+00:00"}
17 {"question": "how many customers from germany?", "step": "sql_generate_start", "timestamp": "2026-05-20T16:37:39.273883+00:00"}
18 {"question": "how many customers from germany?", "step": "sql_generate_complete", "sql": "SELECT COUNT(\\"customerNumber\\") FROM \\"customers\\" WHERE
  \\"country\\" = 'Germany'", "timestamp": "2026-05-20T16:37:40.669269+00:00"}
19 {"question": "how many customers from germany?", "step": "execution_start", "sql": "SELECT COUNT(\\"customerNumber\\") FROM \\"customers\\" WHERE
  \\"country\\" = 'Germany'", "timestamp": "2026-05-20T16:37:40.669983+00:00"}
20 {"question": "how many customers from germany?", "step": "execution_complete", "success": false, "retry_count": 3, "error": "(psycopg2.OperationalError)
  connection to server at \\"db\\" (172.25.0.2), port 5432 failed: FATAL: password authentication failed for user \\"postgres\\"\\n\\n(Background on this error
  at: https://sqlalche.me/e/20/e3q8)", "row_count": 0, "timestamp": "2026-05-20T16:37:44.747858+00:00"}
21 {"question": "how many customers from germany?", "step": "workflow_complete", "success": false, "latency_ms": 6874.07, "summary": "I was unable to
  answer your question after 3 attempts. Last error: (psycopg2.OperationalError) connection to server at \\"db\\" (172.25.0.2), port 5432 failed: FATAL:
  password authentication failed for user \\"postgres\\"\\n\\n(Background on this error at: https://sqlalche.me/e/20/e3q8)", "timestamp": "2026-05-20T16:37:44.
  750547+00:00"}
```

8. Database Schema

The classicmodels database has 8 tables:

Table	Key Columns
customers	customerNumber, customerName, country, creditLimit
orders	orderNumber, orderDate, status, customerNumber
orderdetails	orderNumber, productCode, quantityOrdered, priceEach
products	productCode, productName, productLine, quantityInStock
productlines	productLine, textDescription
employees	employeeNumber, firstName, lastName, jobTitle, officeCode

offices	officeCode, city, country, territory
payments	customerNumber, checkNumber, amount, paymentDate

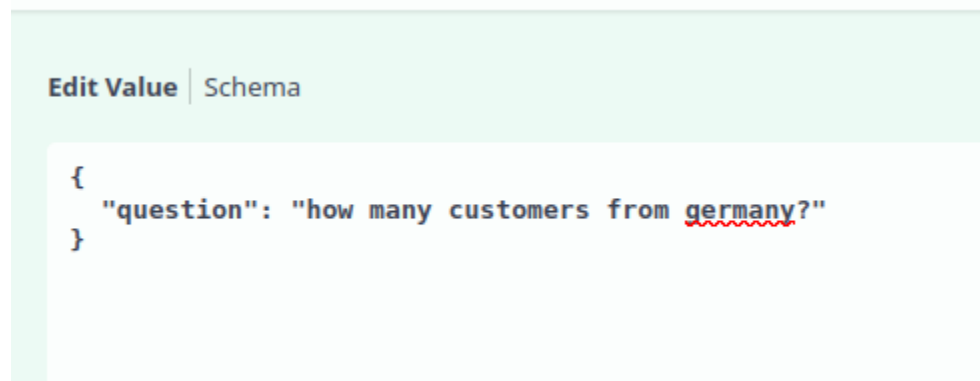
Important: All column and table names use camelCase and require double quotes in PostgreSQL queries.

9. Security Rules

The validator blocks all of the following — only SELECT is allowed:

- INSERT, UPDATE, DELETE, DROP, TRUNCATE
- ALTER, CREATE, REPLACE, MERGE
- EXEC, EXECUTE, GRANT, REVOKE
- Multiple statements (semicolon-separated)

SWAGGER UI:



Response body

```
{
  "sql": "SELECT COUNT(\"customerNumber\") FROM \"customers\" WHERE \"country\" = 'Germany'",
  "result": [
    {
      "count": 13
    }
  ],
  "summary": "There are **13 customers from Germany** in the database.",
  "status": "success",
  "retry_count": 0,
  "latency_ms": 3805.34,
  "error": null
}
```

STREAMLIT

Mini SQL Agent

Ask questions about the classicmodels database in plain English.

How many customers are in germany?

Germany Customer Count

There are 13 customers located in Germany in the database.

Generated SQL

```
SELECT COUNT("customerNumber") FROM "customers" WHERE "country" = 'Germany'
```

How many shipped orders are from USA customers?

Summary

There are 105 shipped orders from customers located in the USA.

Generated SQL

```
SELECT COUNT("orders"."orderNumber") FROM "orders" JOIN "customers" ON "orders"."customerNumber" = "customers"."customerNumber" WHERE "orders"."status" = 'shipped' AND "customers"."country" = 'USA'
```